PROJECT-BASED LEARNING

PHASE-II EVALUATION

Adaptive Memory Allocation Fragmentation Analyzer

Team ID: DSCPP-III-2026-T012

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T012
Semester: 3rd
Section: A,E,F,H
Domain: *C++,DSA*

Team Members

1. *Vamsi –2510011373*
2. *Shweta Singhal –2510011209*
3. *Akshat Kumar–2510011779*
4. *Parik Maan –2510010904*

Mentor

Mr. Himanshu Mainwal

Interactions completed: _____

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed: A C++-based memory allocation simulator to analyze memory utilization and fragmentation. Designed: System architecture, process management, and memory allocation strategies. Achieved: Implemented First Fit, Best Fit, and Worst Fit algorithms with process simulation and memory management. Outcome: Established the foundation for strategy comparison and fragmentation analysis in Phase-II.

Feedback Received

- *Improve memory allocation accuracy and handling.*
- *Test algorithms on different workloads.*
- *Include fragmentation analysis and performance comparison.*

Changes Incorporated

- *Improved memory allocation and deallocation.*
- *Added basic, fragmentation, and mixed workloads.*
- *Implemented memory statistics and strategy comparison.*

Problem Statement & Finalized Objectives

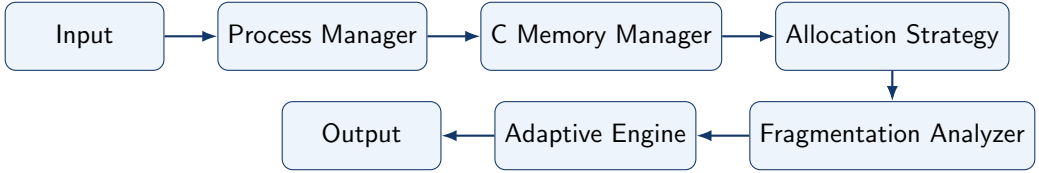
Problem Statement

Operating systems frequently face memory fragmentation and inefficient allocation due to repeated process allocation and deallocation. This project simulates memory management and compares First Fit, Best Fit, and Worst Fit to improve memory utilization and reduce allocation failures.

Finalized Objectives

1. *Memory Allocation: Implement First Fit, Best Fit, and Worst Fit memory allocation algorithms.*
2. *Process Simulation: Simulate process creation, memory allocation, execution, and deallocation.*
3. *Fragmentation Analysis: Analyze memory utilization, free memory, largest free block, and external fragmentation.*
4. *Strategy Comparison: Compare allocation algorithms using the same workloads to evaluate their performance and allocation failures.*

Updated System Architecture / Workflow



Major Components

- *Process Manager*
- *Memory Allocation Module*
- *Fragmentation Analyzer*

Changes Since Phase-I

- *Architecture Change: Simplified the main menu to focus on Process Simulation and Strategy Comparison.*
- *New Component: Added basic, fragmentation, and mixed workloads with integrated allocation and deallocation operations.*
- *Improved Workflow: Enabled automated comparison of First Fit, Best Fit, and Worst Fit using memory utilization, fragmentation.*

Module Design & Functional Requirements

Major Modules

1. *Process Management Module – Manages process creation, arrival time, execution time, and priority.*
2. *Memory Management Module – Handles memory allocation, deallocation, and free memory blocks.*
3. *Allocation Strategy Module – Implements First Fit, Best Fit, and Worst Fit algorithms.*
4. *Fragmentation Analysis Module – Calculates memory utilization, external fragmentation, and allocation failures.*

Functional Requirements

- *Process Simulation: Supports basic, fragmentation, and mixed workloads.*
- *Memory Allocation: Allocates memory using First Fit, Best Fit, and Worst Fit strategies.*
- *Memory Deallocation: Releases memory when processes finish and manages available blocks.*
- *Performance Analysis: Compares strategies using memory utilization, largest free block, fragmentation, and allocation failures.*

Module Dependency

The Process Management Module supplies process details to the Memory Management Module, which works with the Allocation Strategy Module to allocate and release memory. The Fragmentation Analysis Module evaluates the resulting memory state and generates performance statistics for strategy comparison.

Technology Stack & Development Methodology

Technology Stack

- Programming: *C / C++ / Other*
- Database: *File Handling (.txt files)*
- Framework: *Standard C/C++ Libraries, STL*
- Development Tool: *Code::Blocks / Dev-C++/VS Code*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis – Identify system requirements
2. System Design– Design modules and workflow
3. Implementation– Develop allocation algorithms
4. Testing– Test allocation and fragmentation cases
5. Evaluation– Compare performance and memory utilization

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Arrays / Vectors, Linked Lists, Queues, Priority Queue / Heap	Process & Memory Management
Algorithms	Hashing, Searching, Sorting, Greedy Algorithms	Allocation Strategy Analysis
OOPs with C++	Classes, Objects, Encapsulation, Inheritance, Polymorphism	Allocation & Adaptive Engine
C++ Features	STL, File Handling, Exception Handling, Modular Design	Simulation & Reporting
Operating Systems	Processes, Memory Management, Allocation, Deallocation, Fragmentation	Memory Simulation & Analyzer
DBMS / Data Management	File Handling, Data Storage, Basic Data Management	Results & Reports

Key Integration: DSA + C++ + Operating Systems concepts are combined to simulate, analyze, and adapt memory allocation.

Development & Implementation Progress

Module	Status	Work Completed
Process Management	Completed	*Implemented process creation, workload generation, and process tracking.*
Memory Management	Completed	*Implemented memory allocation, deallocation, and free-block management.*
Allocation Strategies	In Progress	*Implemented First Fit, Best Fit, and Worst Fit; testing strategy performance across workloads.*
Fragmentation Analysis	In Progress	*Calculating memory utilization, largest free block, external fragmentation, and allocation failures; validating results.*

Overall Project Completion: *80%*

Working Prototype / Demonstration

Implemented Features

- *Memory Allocation:Implements First Fit, Best Fit, and Worst Fit algorithms.*
- *Process Simulation: Supports basic, fragmentation, and mixed workloads..*
- *Fragmentation Analysis: Tracks memory utilization, free memory, and fragmentation.*
- *Strategy Comparison: Evaluates algorithm performance across the same workload.*

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
First Fit Allocation	*Allocates a process to the first suitable free block.*	*Allocation performed according to First Fit.*	Pass
Best Fit Allocation	*Selects the smallest suitable free block.*	*Allocation performed according to Best Fit.*	Pass
Worst Fit Allocation	*Selects the largest suitable free block.*	*Allocation performed using Worst Fit.*	*Pass*
Strategy Comparison	*Compares all three algorithms on the same workload.*	*Displays memory utilization, fragmentation, and allocation failures.*	*Pass*

Results / Validation

Memory Utilization: Measures the percentage of memory in use.
External Fragmentation: Measures fragmentation across separate free blocks.
Allocation Failures: Counts unsuccessful memory allocation requests.
Performance Comparison: Evaluates First Fit, Best Fit, and Worst Fit using identical workloads.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: *https://github.com/Vamsi-12345/Adaptive-Memory-Allocation-Fragmentation-Analyzer*
- README.md: *Updated*
- Contributors: *Vamsi , Shweta Singhal , Akshat kumar , Parik Maan*
- Repository Status: *Active Development — Code maintained and updated on GitHub.*
- Branches: *Main branch maintained*

Development Activity

- *Regular commits maintained*
- *Meaningful commit messages*
- *Individual contribution visible*
- *Latest implementation uploaded*
- *Documentation maintained*

Requirement: All four team members must be added as GitHub contributors and make individual commits using their own GitHub accounts.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Vamsi	*Team Lead & Integration*	*System architecture, project integration, and strategy comparison*	*13*
Parik Maan	*Memory Management*	*Memory blocks, allocation tracking, deallocation, and free-block management*	*4*
Shweta Singhal	*Allocation Algorithms*	*Implementation of First Fit, Best Fit, and Worst Fit algorithms*	*6*
Akshat Kumar	*Process Management*	*Process creation, workload management, and process simulation*	*8*

Individual Evaluation

Each team member must explain their assigned module, demonstrate their implementation, justify key design decisions, and present relevant testing results to verify their individual contribution.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Technical: Handling memory fragmentation and allocation failures.*
- *Integration: Connecting process management with memory allocation.*
- *Implementation: Ensuring accurate statistics across workloads.*

Solutions / Learning

- *Solution: Tested three allocation algorithms across different workloads.*
- *Learning: Improved understanding of memory management and algorithm analysis.*

Phase-III Roadmap

1. *Complete remaining modules*
2. *Integrate all modules.*
3. *Test and optimize performance.*
4. *Validate results and finalize the prototype.*
5. *Complete documentation.*

Expected Outcomes & References

Expected Final Outcomes

- *Working memory allocation simulator*
- *Fragmentation analysis with measurable results*
- *Comparison of allocation strategies*
- *Practical understanding of memory management*
- *Scope for future optimization and scalability*

References

1. *GeeksforGeeks – Operating System Memory Management concepts*
2. *W3Schools – C Programming Memory Management tutorials*
3. *Programming in ANSI C – E. Balagurusamy, McGraw Hil*
4. *Data Structures Through C – G.S. Baluja*
5. *Bjarne Stroustrup, The C++ Programming Language*

Thank You
Questions & Suggestions